

Agentic Graph Reasoning: Claim-Level Structural Verification for Multi-Hop Knowledge Graph Question Answering

Md. Sakif Khan^a, Sadia Sharmin^a

^aDepartment of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh

Abstract

Keywords: knowledge graph question answering, large language models, agentic retrieval, ablation study, hallucination, benchmark quality

1. Introduction

Large language models answer factual questions fluently, and often enough they answer them wrongly. The errors are systematic rather than random, and they are hard to catch from the output alone, because a fabricated answer reads in the same register as a correct one; Huang et al. [1] treat this as an intrinsic consequence of how the models are built rather than an incidental defect to be patched away. The problem sharpens once an answer requires composing several facts rather than recalling one. Errors compound, since a wrong resolution at the first hop rarely announces itself downstream, and a question-level correctness check never sees the intermediate steps at all — which leaves a system that answers correctly

Email address: `sakifkhanopu@gmail.com` (Md. Sakif Khan)

by accident indistinguishable from one that answers correctly by sound reasoning.

Retrieval-augmented generation [2] addresses the first half of this by grounding the model in retrieved text. It does not address the second. Retrieval runs once, before reasoning begins, so a multi-hop question must be answered from whatever that single query returned, and the passage holding the *intermediate* entity need not resemble the original question — which is the only signal the retriever has to rank on. Representing the corpus as a knowledge graph [3] makes composition a traversal rather than an inference over prose, and GraphRAG [4] exploits this, but it still retrieves statically: a neighbourhood fetched in one shot cannot know what the next reasoning step will need, and is either too small to contain the answer or so large that the relevant edges are buried.

Agentic systems close that gap by interleaving the two: the model takes one step, observes what it found, and decides where to look next [5, 6, 7]. Retrieval becomes navigation, and the system becomes an agent operating on the graph rather than a pipeline reading from it. Section 2 traces that line of work in detail.

One property survives this progression unchanged. Every system named above emits whatever its final generation call produces from the traversed context, and nothing checks that what the answer *asserts* is supported by what the agent actually retrieved. The gap is not fabrication — our measurements confirm that graph-navigating systems assert entities that exist in the graph and are reachable along the paths they walked (Section 5.4). It is that an entity can sit in the retrieved subgraph, have been retrieved correctly,

and still be the wrong answer: an intermediate node from the reasoning chain, a container where the question asked for a member, or one of several plausible neighbours chosen arbitrarily. A system that verifies retrieval but never verifies assertion cannot catch either case.

This paper presents **Agentic Graph Reasoning** (AGR), a knowledge-graph question-answering framework built as an explicit state machine over a constrained, deterministic graph tool API, in which the program orchestrates the loop and the language model is consulted only at bounded decision points. Its distinguishing component is a *Structural Verification Layer*: before an answer is emitted, the draft is decomposed into atomic claims and each is checked against the triples the agent actually traversed. Claims that cannot be grounded trigger targeted re-exploration or are removed, and every answer AGR returns is paired with the supporting triples that justify it.

We are precise about what that layer buys, because the ablation of Section 6 is precise about what it does not. Claim verification does not measurably change how often AGR is right; it changes how often AGR is right *for a reason it can exhibit*. The contribution is the output contract — an answer together with the traversed evidence for it — rather than an accuracy gain, and Section 6 reports the null that establishes this at the same length as the effects it does detect.

On 400 certified questions from each of WebQSP [8] and ComplexWeb-Questions [9], over a Freebase-derived environment of 2.59 million entities and 8.31 million triples, AGR reaches Hits@1 of 0.755 and 0.522 against four baselines sharing one frozen backbone and one call budget, and it does so at roughly half the language-model calls of the strongest of them. On

ComplexWebQuestions it is the only system whose accuracy rises with hop count. Its advantage over the agentic baseline is located rather than asserted: on the questions where the shared cap lets that baseline finish it is slightly ahead, and the entire aggregate margin comes from the 29% and 44% of questions on which the cap truncates it.

Contributions.. This paper contributes:

1. **AGR**, a state-machine agent over a constrained graph tool API whose answers carry the traversed triples that support them, so the reasoning is auditable rather than merely asserted (Section 3).
2. A **Structural Verification Layer** that checks atomic claims against traversed adjacency before emission, with its repair routes and their measured firing rates reported as they behave rather than as designed (Section 3.2).
3. A **controlled comparison** against four baselines on one frozen backbone under one call budget, so that differences are attributable to architecture rather than to model capacity (Section 5).
4. A **component-level ablation** with paired significance testing, which finds that removing the planner *improves* accuracy on the shallower benchmark while reducing token use, and that three of four components show no detectable effect (Section 6).
5. A **hand-read census** of all 259 remaining failures, naming the *echo attractor* — a true, grounded entity one hop from the answer that every grounding check passes — and quantifying label defects in both benchmarks (Section 7).

Section 2 positions AGR against prior work. Section 3 specifies the framework and the verification layer, Section 4 the environment and experimental protocol, and Section 5 the measurements. Section 6 attributes them to components, Section 7 reads what remains, and Section 8 states the limits.

2. Related Work

2.1. Static Retrieval over Graphs

Retrieval-augmented generation [2] attaches a language model to an external corpus by embedding the question, retrieving lexically or semantically similar passages, and letting the model answer from them. Where the answer sits in a single retrievable passage this works well; where it must be composed across passages, the ranking signal is measuring the wrong thing, because the passage carrying the intermediate entity resembles neither the question nor anything the system yet knows to look for. Surveys of the graph–language-model intersection identify this as the central limitation of flat retrieval [10].

Knowledge graphs make the missing structure explicit, storing facts as typed edges between identified entities so that composition becomes a traversal rather than an inference over prose. GraphRAG [4] builds a graph over a corpus and retrieves structured context in place of passages, and related systems extend the idea to long-term memory [11] and temporally-qualified facts [12]. All of them retrieve *statically*: a subgraph is fetched around the linked entities in one shot and handed to the model. Because retrieval runs first and runs once, it cannot know what the second hop will need, and a fixed-radius neighbourhood is either too small to contain the answer or so

large that the relevant edges are lost among thousands of irrelevant ones.

2.2. *Agentic Navigation*

Interleaving retrieval with reasoning removes that constraint. The model takes one step, observes what it found, and decides where to look next; retrieval becomes navigation and the system becomes an agent operating on the graph rather than a pipeline reading from it. This inherits the tool-use and self-reflection patterns established for language agents generally [13, 14, 15].

Think-on-Graph [5] established the pattern for knowledge graphs with model-guided beam search over relations and entities. Plan-on-Graph [6] added two mechanisms to it: decomposition into sub-objectives, and backtracking driven by reflection. Reasoning-on-Graphs [7] takes a different route, generating grounded relation paths as explicit plans and then traversing them. Others add tool abstractions [16], cope with incomplete graphs [17], or generalise across structured sources [18], and recent surveys treat the convergence as a research programme in its own right [19, 20].

KBQA-o1 [21] is the closest system to AGR’s search strategy and the most important to distinguish from it. It performs agentic knowledge base question answering with full Monte Carlo Tree Search — a policy model proposing actions, a reward model scoring them, and rollouts whose values propagate back through the tree — around a ReAct-style agent generating logical forms against Freebase, with the exploration additionally producing annotations for incremental fine-tuning. AGR does not do this. Its search is guided best-first with a bounded beam and backtracking, and it carries neither rollouts nor value backpropagation; the two are separable on that

basis, and we name the difference rather than blur it.

2.3. *Verification and Self-Correction*

A parallel line checks generated content after the fact. Chain-of-Verification [22] has a model draft a response, plan verification questions about it, answer those independently, and regenerate; multi-agent debate [23] achieves a related effect by having several instances critique one another. Both establish the premise this paper’s verification layer rests on — that decomposing a draft and checking its parts reduces factual error — but both check against the model’s own knowledge rather than an external symbolic source, and the limits of models correcting themselves without such a source are documented [24]. Hu et al. [25] demonstrate a self-correcting agentic graph pipeline in a clinical setting, and FActScore [26] decomposes generations into atomic facts for evaluation, which is the same decomposition applied as a metric rather than as a control step.

What none of them provides is a claim-level check against the *traversed* triples, performed *before* the answer is emitted, inside a graph-navigating agent. That is the gap Section 3.2 fills.

2.4. *The Attribution Gap*

A second gap is methodological. Agentic systems are assemblies — decomposition, scoring, backtracking, self-correction — and the literature reports them as wholes. When such a system outperforms a static baseline, the published work rarely establishes which mechanism produced the gain, or whether some contribute nothing. Later work then inherits the entire assembly on faith, along with its cost.

The exception is instructive. KBQA-o1’s own ablation removes MCTS while leaving every other component intact and drops GrailQA F1 from 78.5 to 48.5 [21] — a thirty-point attribution to a single mechanism, obtained precisely because the authors ablated rather than reporting the assembly whole. Section 6 applies the same instrument to AGR, and reports the components for which it detects nothing at the same length as the one for which it does.

3. The AGR Framework

AGR inverts the usual agent pattern. Rather than handing the model a set of tools through native function-calling and letting it drive the loop itself, a program orchestrates every step, calls the graph tools deterministically, and consults the model only at bounded decision points through prompts constrained to return JSON. Three consequences follow directly. Every graph operation is logged by construction rather than reconstructed from the model’s account of what it did. Budgets are enforced by counters a router checks, not requested in a prompt the model may ignore. And a malformed model response degrades one decision rather than corrupting control flow: every call is wrapped in a parser that, on failure, spends one metered retry asking the model to repair its own output before falling back. What follows specifies the loop that produces a traversal (Section 3.1), the layer that checks a draft against it before an answer is emitted (Section 3.2), and the budgets that bound the whole loop’s cost (Section 3.3).

3.1. State Machine and Tool API

Figure 1 gives the topology: six agent nodes over a shared typed state, with two cycles. The **Planner** decomposes the question into an ordered chain of sub-objectives in one model call; **Explorer** and **Evaluator** form a search loop, expanding the frontier and judging whether the active sub-objective is satisfied; **Backtracker** restores an earlier, higher-scoring frontier and bans the edges that led away from it, which is what turns backtracking into a search operation rather than a deterministic no-op against an unchanged scorer. Two properties of the topology matter beyond the nodes themselves. It contains cycles rather than running once through, because exploration continues until the evaluator is satisfied and verification can send the agent back to explore. And every path from Evaluator to Answerer passes through the **Verifier** — there is no route that reaches the Answerer without it, which is the structural expression of this paper’s contribution and the subject of Section 3.2.

Five graph operations back this loop, specified as parameterised Cypher templates rather than left to free-form generation the model would write itself — a generated-query failure is indistinguishable, in the results, from a reasoning failure, and this paper’s central claims are about reasoning and verification, not about syntax. Only four of the five are reachable from a node in the final design. `verify_triple` was built first, to check a claim $\langle h, r, t \rangle$ by asking whether that exact relation held between the two entities; it did not survive contact with generated text, because claim decomposition phrases relations in English (“played for”, “is the capital of”) that rarely correspond to a single Freebase predicate, and matching on them rejected claims that

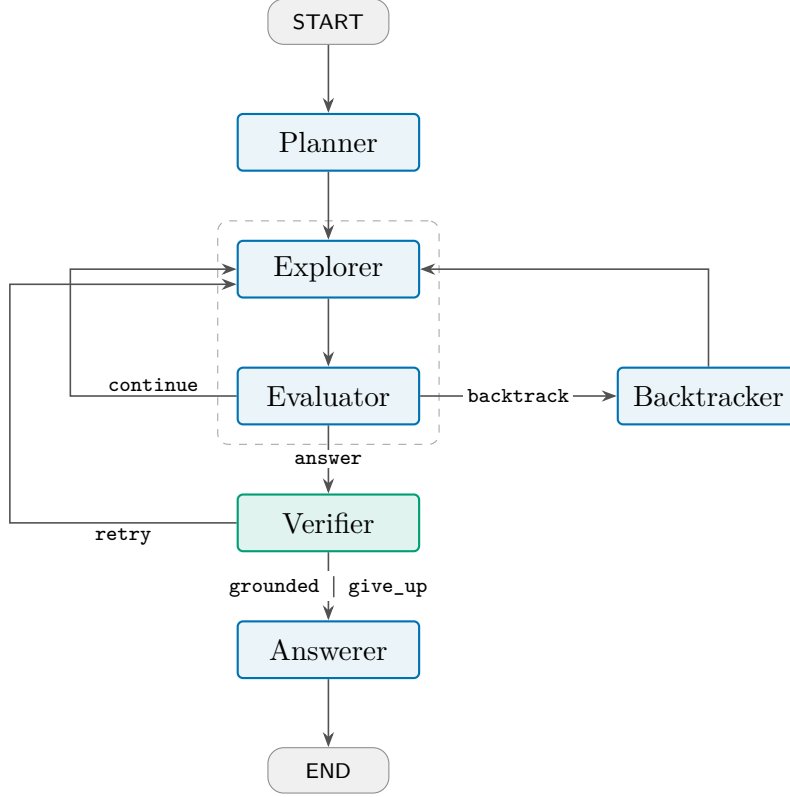


Figure 1: The AGR state machine. The dashed region is the Explorer $\leftrightarrow$ Evaluator search loop; Verifier $\rightarrow$ Explorer is verification-driven re-exploration. Both cycles are bounded by the budgets of Section 3.3. Every path from Evaluator to Answerer passes through the Verifier.

were true. `verify_connection` replaced it: it drops the relation and asks only whether the two entities are adjacent, directly or through one mediator node, which is the check Section 3.2 actually uses. `verify_triple` remains in the API and in its unit tests, but nothing in the running loop calls it.

Each candidate edge the Explorer considers is scored by

$$\text{score}(c) = \alpha \cos(v_{\text{rel}(c)}, v_{\text{obj}}) + (1 - \alpha) \text{LLM}(c),$$

blending the cosine similarity of the candidate relation’s embedding against the embedding of the *active sub-objective* — not the original question, which is the concrete mechanism through which decomposition is meant to help — with a batched model judgement over a pre-filtered shortlist of twenty, at the frozen $\alpha = 0.7$. The blend is applied after the model call rather than inside the prompt, so a prompt free of any configuration value lets an α sweep reuse one set of cached responses across every condition; setting $\alpha = 1$ disables the model term entirely, which is the ablation Section 6 reports.

3.2. Structural Verification Layer

The navigation loop above ends with a set of traversed triples and a set of accumulated candidate entities; nothing in it constrains what the final answer *asserts*. The verifier is the check that does. A single model call drafts prose, extracts the entities the draft asserts as its answer, and decomposes the draft into atomic claims $\langle h, r, t \rangle$ over entity *names*, since the model has never seen a graph identifier; the relation slot is left as free text, because asking the model to name a Freebase predicate would fail as unreliably as `verify_triple` did.

Each claim is routed through up to three tests, and a claim leaves at the first one that settles it. **Traversed adjacency** collapses the traversed triples into an undirected set of endpoint pairs; a claim is supported if its pair is in that set, and — uniquely among the three routes — every traversed triple joining the pair is attached as citable evidence. **verify_connection** catches claims whose endpoints were both reached but never directly walked together, querying the full graph for the same relation-agnostic adjacency. Claims that clear neither route, together with those naming an entity the agent

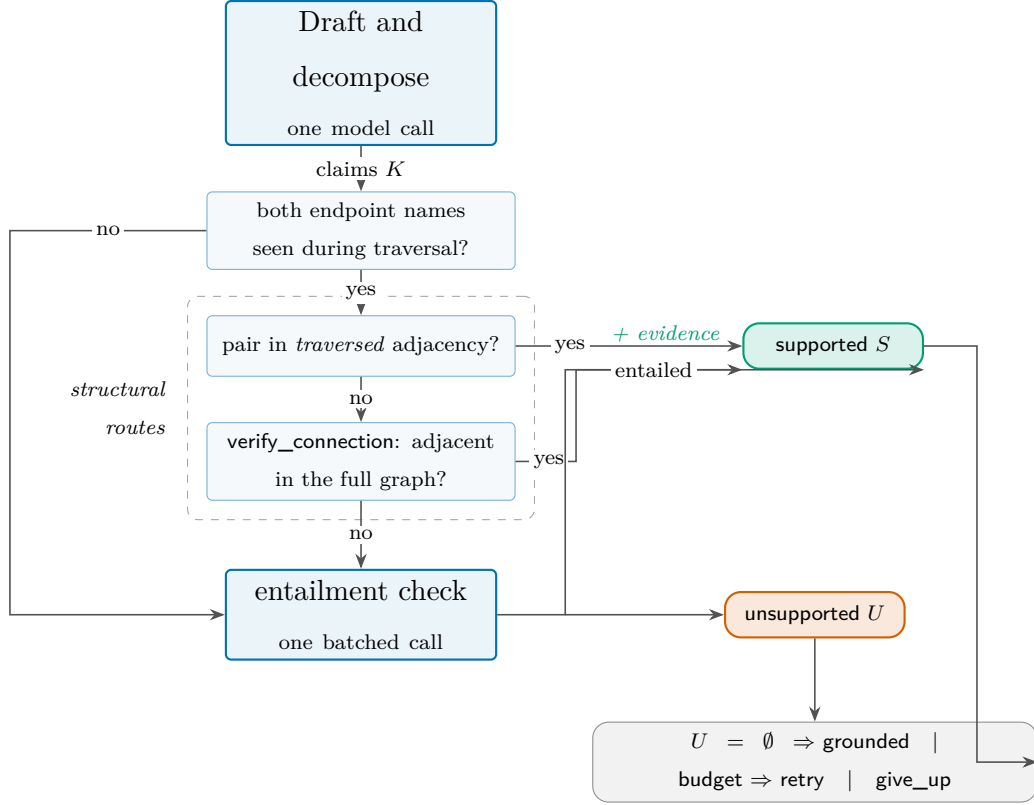


Figure 2: The path one atomic claim takes through the verification layer. Three tests are tried in a fixed, cheap-to-expensive order and a claim leaves at the first that settles it. Only traversed adjacency attaches citable evidence; a claim naming an entity the agent never reached skips both structural routes entirely.

never traversed at all, fall to an **entailment check**: one batched model call judging each remaining claim against the same fact window the drafter saw. The order is deliberate rather than incidental: symbols decide and the model is consulted only where the graph is silent, so cheap-to-expensive ordering also means the model adjudicates a residue rather than every claim, and only the first route yields evidence that can be attached to the answer.

The router that follows has three outcomes. No unsupported claim routes to **grounded**, and the draft is emitted verbatim with no further model call. An unsupported claim with verification, depth, and time budget remaining routes to **retry**: the verifier re-targets the agent on the first unsupported claim and control returns to the Explorer. Otherwise the router takes **give_up**, and one rewrite call restates the draft around a *deterministically filtered* entity list — an entity survives if it appears in a supported claim or in no unsupported claim — rather than letting the model regenerate the list itself, which an earlier design did and which reintroduced exactly the ungroundedness the layer exists to prevent.

The layer’s reach should be stated plainly here, because it bears directly on how the null result of Section 6 should be read. On the 80-question development set used to validate this design, 77 questions asserted no unsupported claim and took the grounded path at zero additional model cost; the remaining 3 took **give_up**, and the retry route did not fire even once, because every question that reached the verifier with an unsupported claim had already exhausted its depth budget by the time it got there. On most questions, in other words, the layer is a pass-through that certifies a draft rather than a mechanism that visibly intervenes. Sections 5 and 6 report

Table 1: Budget configuration. Every row is hashed into the run configuration, so a reported result can always be traced to the caps that produced it.

Parameter	Value	Enforced in
max_depth	4	post-evaluation <i>and</i> post-verification routers
beam_width	3	Explorer (per-anchor slice)
max_anchors	5	Explorer <i>and</i> Evaluator (frontier slice)
max_backtracks	3	post-evaluation router
max_llm_calls	25	model client, before every call
max_verify_iters	2	post-verification router <i>and</i> the verifier
max_seconds	300	post-evaluation <i>and</i> post-verification routers

what it measurably changes at test scale; engineering detail beyond that — defects found along the way and recorded rather than quietly repaired — is in the repository rather than reproduced here.

3.3. Budgets and Termination

Table 1 lists the caps that bound every run. They are enforced in routers and the model client rather than requested in prompts, and three of them — depth, wall-clock, and verification iterations — are checked at two separate sites each, because the `retry` edge re-enters the Explorer without passing back through the router that would otherwise catch it; closing that gap is what lets Table 1 be called a guarantee rather than a convention.

Every cycle in Figure 1 therefore passes through a router that checks or decrements a monotone counter, so no sequence of model outputs, however

adversarial, can produce a non-terminating run. The cost of enforcing this is small in the common case: the grounded path above adds nothing beyond the single drafting call already required to produce a draft. Section 5.2 reports what the layer costs in aggregate, on the questions where it is not a pass-through.

4. Experimental Setup

4.1. Knowledge Environment

4.2. Benchmarks, Baselines, and Backbone

4.3. Metrics and Protocol

5. Results

5.1. Accuracy

5.2. Cost and the Location of the Margin

5.3. Accuracy by Hop Depth

5.4. Groundedness

6. Component-Level Attribution

6.1. When Planning Hurts

6.2. Three Components with No Detected Effect

7. Error Analysis

7.1. The Echo Attractor

7.2. Benchmark Label Defects

8. Discussion and Limitations

9. Conclusion

Declaration of competing interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported here.

Data availability

Code, prompts, per-question run records, and the analysis scripts that produce every figure and table in this paper are available at <https://github.com/sakif-khan/agentik-graph-reasoning> under the MIT licence. The knowledge environment is derived from Freebase [3] and is rebuilt from its public source by the scripts in that repository rather than redistributed; the question sets are the public WebQSP [8] and ComplexWebQuestions [9] releases.

References

- [1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, *ACM Transactions on Information Systems* 43 (2) (2025) 1–55.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 9459–9474.
- [3] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, J. Taylor, Freebase: A collaboratively created graph database for structuring human knowledge, in: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, 2008, pp. 1247–1250.

- [4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization, arXiv preprint arXiv:2404.16130 (2024).
- [5] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, J. Guo, Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2307.07697.
- [6] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, H. Xiong, Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, arXiv:2410.23875.
- [7] L. Luo, Y.-F. Li, G. Haffari, S. Pan, Reasoning on graphs: Faithful and interpretable large language model reasoning, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2310.01061.
- [8] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, J. Suh, The value of semantic parse labeling for knowledge base question answering, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), 2016, pp. 201–206.
- [9] A. Talmor, J. Berant, The web as a knowledge-base for answering complex questions, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, pp. 641–651.

- [10] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, J. X. Yu, A survey of graph meets large language model: Progress and future directions, arXiv preprint arXiv:2311.12399 (2023).
- [11] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su, HippoRAG: Neurobiologically inspired long-term memory for large language models, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, pp. 59532–59569.
- [12] D. Li, Y. Niu, Y. Ai, X. Zou, B. Qi, J. Liu, T-GRAG: A dynamic GraphRAG framework for resolving temporal conflicts and redundancy in knowledge retrieval, in: Proceedings of the 33rd ACM International Conference on Multimedia, 2025, pp. 11880–11889.
- [13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.
- [14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.
- [15] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.
- [16] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, J.-R. Wen, KG-

Agent: An efficient autonomous agent framework for complex reasoning over knowledge graph, arXiv preprint arXiv:2402.11163 (2024).

- [17] Y. Xu, S. He, J. Chen, Z. Wang, Y. Song, H. Tong, K. Liu, J. Zhao, Generate-on-graph: Treat LLM as both agent and KG for incomplete knowledge graph question answering, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, arXiv:2404.14741.
- [18] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, J.-R. Wen, StructGPT: A general framework for large language models to reason over structured data, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, arXiv:2305.09645.
- [19] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, IEEE Transactions on Knowledge and Data Engineering 36 (7) (2024) 3580–3599.
- [20] C. Ma, Y. Chen, T. Wu, A. Khan, H. Wang, Unifying large language models and knowledge graphs for question answering: Recent advances and opportunities, in: Proceedings of the 28th International Conference on Extending Database Technology (EDBT), 2025, pp. 1174–1177.
- [21] H. Luo, H. E, Y. Guo, Q. Lin, X. Wu, X. Mu, W. Liu, M. Song, Y. Zhu, L. A. Tuan, KBQA-o1: Agentic knowledge base question answering with monte carlo tree search, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025, arXiv:2501.18922.

- [22] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, J. Weston, Chain-of-verification reduces hallucination in large language models, in: Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 3563–3578.
- [23] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: Proceedings of the 41st International Conference on Machine Learning, 2024.
- [24] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, D. Zhou, Large language models cannot self-correct reasoning yet, in: International Conference on Learning Representations (ICLR), 2024.
- [25] Y. Hu, W. Xuan, Q. Zhou, Z. Li, Y. Li, J. Hu, F. Fang, A self-correcting agentic graph RAG for clinical decision support in hepatology, *Frontiers in Medicine* 12 (2025) 1716327.
- [26] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 12076–12100.